

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
Кафедра «Информационная безопасность в вычислительных системах и сетях»

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

к практическому занятию № 4

«Стек, очередь и дек»

по дисциплине «Алгоритмы и структуры данных»

Параметр	Значение
Направление подготовки	03.03.01 Прикладные математика и физика
Направленность	03.03.01 Прикладные физика и математика
Семестр	7
Продолжительность	2 академических часа
Компетенция / индикатор	ПК-4 / ПК-4.4
Статус	Методические указания

Ростов-на-Дону
2026

1. Паспорт практического занятия

Основание разработки. Актуальная РПД дисциплины «Алгоритмы и структуры данных» на 2026/2027 учебный год, раздел 2 «Линейные структуры данных и абстрактные типы данных»: стек, очередь, дек, представление данных в памяти и амортизированный анализ.

Преимственность. В ПР №3 были реализованы динамический массив и односвязный список. В этой работе внутреннее представление отделяется от публичного контракта: одна и та же спецификация очереди может быть реализована кольцевым массивом или связным списком.

Цель. Научиться выбирать стек, очередь или дек по порядку доступа, реализовывать операции с заданными гарантиями и доказывать сохранение инвариантов структуры.

Постановка задачи. Реализовать стек, кольцевую очередь и ограниченный дек; проверить контракты операций; решить демонстрационную задачу потоковой обработки и исследовать фактическую стоимость операций.

1.1. Планируемые результаты

- различать семантику LIFO, FIFO и доступа с обоих концов;
- формулировать контракт push/pop, enqueue/dequeue, append/appendleft и соответствующих операций удаления;
- реализовывать кольцевой буфер без линейного сдвига элементов;
- доказывать сохранение инвариантов размера, индексов и порядка элементов;
- получать худшие и амортизированные оценки времени и памяти;
- выбирать АД для обработки журналов, событий и скользящих окон;
- проводить воспроизводимый эксперимент, отделяя проверку корректности от профилирования.

Уровень	Деятельность	Проверяемый артефакт
Базовый	Корректные операции стека и очереди	Код и тесты границ
Средний	Кольцевой буфер и доказательство инварианта	Псевдокод, $T(n)$, $M(n)$
Продвинутый	Дек в задаче скользящего окна	Решение, CSV и анализ
ПК-4 / ПК-4.4	Выбор и программная реализация АД	Отчёт, код, защита

2. Инструменты и технологии

- Python 3.12 или новее; collections.deque используется как эталон и прикладной инструмент, но не вместо собственной кольцевой очереди;
- dataclasses и typing для явного описания узлов и контрактов;
- time.perf_counter_ns, statistics.median, tracemalloc, gc и csv для эксперимента;
- pytest допускается; обязательный минимальный набор assert-тестов должен запускаться без сторонних библиотек;
- VSCodium, Visual Studio Code, PyCharm, JupyterLab либо командная строка.

Ограничения. Не используйте list.pop(0) как эффективную очередь: операция сдвигает оставшиеся элементы. Начинайте серию с малых n , формируйте вход вне таймера и измеряйте память отдельно от времени.

3. Краткие теоретические сведения

3.1. Абстрактный тип данных

АТД задаёт допустимые значения и операции, но не фиксирует внутреннее представление. Стек может быть реализован динамическим массивом или списком; очередь — связной цепочкой либо кольцевым буфером. Клиентский алгоритм должен зависеть от контракта, а не от способа хранения.

АТД	Порядок	Основные операции	Типичный сценарий
Стек	LIFO	push, pop, peek	вложенность, возврат, отмена
Очередь	FIFO	enqueue, dequeue, front	заявки, BFS, моделирование
Дек	оба конца	append/appendleft, pop/popleft	окна, двусторонняя обработка

3.2. Контракты и ошибки

- push/enqueue добавляет один элемент и увеличивает size на 1;
- pop/dequeue возвращает и удаляет ровно один элемент; на пустой структуре возбуждается IndexError;
- peek/front возвращает элемент без изменения size;
- для ограниченного буфера заранее фиксируется политика переполнения: ошибка, перезапись старого или расширение;
- операция не должна оставлять структуру в частично изменённом состоянии при ошибке.

3.3. Кольцевая очередь

Буфер ёмкости capacity хранит size логических элементов. Индекс head указывает на первый элемент, а физическая позиция логического элемента i вычисляется как $(head + i) \bmod capacity$. При удалении head сдвигается по модулю capacity; элементы не копируются.

- $0 \leq size \leq capacity$;
- если $size > 0$, front расположен в $data[head]$;
- логический порядок задаётся позициями $(head + i) \bmod capacity$ для $0 \leq i < size$;
- позиция добавления равна $(head + size) \bmod capacity$;
- после расширения $head = 0$, а элементы занимают $data[0 \dots size-1]$ в прежнем логическом порядке.

3.4. Дек и монотонная очередь

Дек обеспечивает $O(1)$ на обоих концах. Для максимума скользящего окна в деке хранят индексы кандидатов в порядке убывания значений. Индексы вне окна удаляются слева, а кандидаты не больше нового элемента — справа. Каждый индекс добавляется и удаляется не более одного раза, поэтому весь алгоритм имеет $\Theta(n)$.

3.5. Сложность и представление

Операция	Массивный стек	Кольцевая очередь	collections.deque
Добавление	амортизированно $\Theta(1)$	амортизированно $\Theta(1)$	$\Theta(1)$ на концах
Удаление	$\Theta(1)$ с конца	$\Theta(1)$ с начала	$\Theta(1)$ на концах
Просмотр	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$ на концах
Доступ по индексу	$\Theta(1)$, но не часть АТД	не требуется	не предназначен для

Операция	Массивный стек	Кольцевая очередь	collections.deque
		контрактом	общего доступа
Память	$\Theta(n)$ плюс резерв	$\Theta(n)$ плюс резерв	$\Theta(n)$, блочная реализация

4. Пошаговый план выполнения

1. Запишите контракты операций и политику ошибок для пустой структуры и переполнения.
2. Реализуйте стек поверх динамического массива; проверьте LIFO-порядок.
3. Реализуйте кольцевую очередь с head, size и capacity; не используйте pop(0).
4. Добавьте расширение буфера с сохранением логического порядка элементов.
5. Сформулируйте инварианты и покажите их сохранение после enqueue и dequeue.
6. Проверьте пустой буфер, один элемент, заполнение до границы, переход индекса через конец массива и расширение при head $\neq$ 0.
7. Реализуйте прикладную задачу своего варианта со стеком, очередью или деком.
8. Подготовьте независимую проверку на малых данных или сравнение с collections.deque.
9. Измерьте последовательность операций при шести и более размерах, используя прогрев и минимум семь повторов.
10. Сохраните сырые данные, медианы и пики памяти; объясните выбросы и ограничения.
11. Оформите отчёт, исходный код и CSV; раскройте использование внешнего кода и генеративного ИИ.

5. Демо-пример: кольцевая очередь измерений

5.1. Условие и контракт

Датчик выдаёт значения по одному. Требуется хранить их в очереди с динамическим расширением и извлекать в порядке поступления. Пустая очередь допустима; dequeue и front на пустой очереди возбуждают IndexError. После любой операции последовательность извлечения совпадает с порядком ранее добавленных и ещё не извлечённых значений.

5.2. Реализация кольцевой очереди

```
class RingQueue:
    def __init__(self, capacity=4):
        if capacity < 1:
            raise ValueError("capacity must be positive")
        self._data = [None] * capacity
        self._head = 0
        self._size = 0

    def __len__(self):
        return self._size

    @property
    def capacity(self):
        return len(self._data)

    def _resize(self):
```

```

new_data = [None] * (2 * self.capacity)
for i in range(self._size):
    new_data[i] = self._data[(self._head + i) % self.capacity]
self._data = new_data
self._head = 0

def enqueue(self, value):
    if self._size == self.capacity:
        self._resize()
    tail = (self._head + self._size) % self.capacity
    self._data[tail] = value
    self._size += 1

def front(self):
    if self._size == 0:
        raise IndexError("front from empty queue")
    return self._data[self._head]

def dequeue(self):
    value = self.front()
    self._data[self._head] = None
    self._head = (self._head + 1) % self.capacity
    self._size -= 1
    if self._size == 0:
        self._head = 0
    return value

```

5.3. Функциональная проверка

```

from collections import deque

q = RingQueue(2)
ref = deque()
for value in (10, 20, 30, 40):
    q.enqueue(value); ref.append(value)
assert len(q) == 4 and q.capacity == 4
assert q.front() == 10
assert [q.dequeue(), q.dequeue()] == [ref.popleft(), ref.popleft()]
q.enqueue(50); q.enqueue(60) # переход через конец физического буфера
while ref:
    assert q.dequeue() == ref.popleft()
assert len(q) == 0
try:
    q.dequeue()
    raise AssertionError("IndexError expected")
except IndexError:
    pass

```

5.4. Доказательство корректности

Инициализация. `size = 0` и `head = 0`, поэтому диапазон логических элементов пуст, а все инварианты выполнены.

Сохранение при enqueue. Новый элемент записывается в первую свободную логическую позицию $(\text{head} + \text{size}) \bmod \text{capacity}$, затем `size` увеличивается на 1. Порядок ранее существовавших элементов не меняется. При расширении они предварительно копируются в том же логическом порядке.

Сохранение при dequeue. Возвращается `data[head]`, после чего `head` переходит к следующей логической позиции, а `size` уменьшается. Остальные элементы сохраняют относительный порядок.

Завершение. Каждая публичная операция содержит конечное число действий; `resize` выполняет ровно `size` копирований. Следовательно, операции завершаются.

5.5. Эксперимент

Фактор	Решение
Размеры	n = 1 000, 2 000, 4 000, 8 000, 16 000, 32 000 и 64 000
Нагрузки	n enqueue; чередование enqueue/dequeue; полный цикл enqueue + dequeue
Сравнение	RingQueue, collections.deque и list с pop(0) как заведомо неэффективная база
Повторы	1 прогревочный и не менее 7 измеряемых; сохранять сырые значения
Итог	медиана времени, пик памяти, отношения при удвоении n
Контроль	одинаковая последовательность операций и контрольная сумма результатов

6. Дополнительный демо-фрагмент: максимум скользящего окна

Фрагмент показывает применение дека. Он дополняет основную демонстрацию, но не заменяет индивидуальное задание.

```
from collections import deque

def window_max(values, k):
    if k <= 0:
        raise ValueError("k must be positive")
    q, result = deque(), []
    for i, value in enumerate(values):
        while q and q[0] <= i - k:
            q.popleft()
        while q and values[q[-1]] <= value:
            q.pop()
        q.append(i)
        if i + 1 >= k:
            result.append(values[q[0]])
    return result

assert window_max([2, 1, 3, 4, 0, 5], 3) == [3, 4, 4, 5]
```

7. Индивидуальные задания для домашней работы

Для своего варианта реализуйте требуемый АТД или прикладной алгоритм, сформулируйте инвариант, подготовьте не менее пяти тестов и проведите эксперимент. Основной демо-вариант и дополнительный демо-фрагмент нельзя сдавать без существенного расширения.

№	Вариант	Условие
1	Вычислитель постфиксной записи	Реализовать стек и вычисление выражения с +, -, × и /. Обработать недостаток операндов и деление на ноль.
2	Проверка вложенных тегов	Стеком проверить корректность пар открывающих и закрывающих тегов; вернуть позицию первой ошибки.
3	Очередь лабораторных заявок	Реализовать ограниченную FIFO-очередь с отказом при переполнении и собрать статистику ожидания.
4	Буфер телеметрии	Реализовать кольцевой буфер фиксированного размера с перезаписью самого старого значения.
5	Минимум скользящего	Деком получить минимумы всех окон длины k за $\Theta(n)$; проверить k = 1,

№	Вариант	Условие
	окна	$k = n$ и повторяющиеся значения.
6	История отмены и повтора	Использовать два стека для undo/redo; новая команда после undo должна очищать стек redo.
7	Очередь на двух стеках	Реализовать enqueue и dequeue двумя стеками и обосновать амортизированную $\Theta(1)$.
8	Стек с минимумом	Поддерживать push, pop и get_min за $\Theta(1)$, корректно обработать повторяющиеся минимумы.
9	Дек команд робота	Добавлять приоритетные команды слева, обычные справа и выполнять слева; проверить детерминированность порядка.
10	Моделирование обслуживания приборов	Очередью смоделировать один обслуживающий канал; вычислить среднее и максимальное ожидание.

8. Требования к отчёту

- номер варианта, цель, вход, выход и ограничения;
- контракт операций и выбранная политика ошибок или переполнения;
- внутреннее представление и инварианты;
- псевдокод ключевых операций и доказательство сохранения инварианта;
- оценки худшего и амортизированного времени, рабочей и результирующей памяти;
- исходный код и минимум пять функциональных тестов;
- протокол измерений, сырые времена, медианы и отдельная серия памяти;
- сравнение с альтернативной структурой или эталоном;
- вывод, не менее трёх ограничений эксперимента и условия применимости;
- версия Python, ОС, команда запуска и раскрытие внешнего кода или ИИ.

Компонент	Баллы	Критерий полного выполнения
Контракт и инварианты	20	Операции и ошибки заданы; сохранение инвариантов обосновано
Реализация АД	25	Порядок обработки корректен, нет скрытых линейных сдвигов
Тесты	15	Не менее пяти тестов, включая пустую структуру и переход границы
Сложность	15	Обоснованы худшие и амортизированные время и память
Эксперимент	15	Есть сырые данные, медианы, контроль результата и память
Отчёт и защита	10	Выбор АД аргументирован, запуск воспроизводим

9. Вопросы для самоконтроля

1. Чем АД отличается от конкретной структуры хранения?
2. Как LIFO, FIFO и доступ с обоих концов определяют выбор структуры?
3. Почему `list.pop(0)` имеет $\Theta(n)$?

4. Какие условия составляют инвариант кольцевой очереди?
5. Как вычисляется физическая позиция логического элемента?
6. Почему после расширения удобно установить $\text{head} = 0$?
7. Как доказать сохранение порядка при `enqueue` и `dequeue`?
8. В чём различие политики расширения, отказа и перезаписи при переполнении?
9. Почему последовательность n добавлений остаётся $\Theta(n)$, хотя отдельное расширение стоит $\Theta(n)$?
10. Когда связная очередь предпочтительнее кольцевого массива?
11. Как дек позволяет получить максимум окна за $\Theta(n)$?
12. Почему каждый индекс в монотонном деке удаляется не более одного раза?
13. Как реализовать очередь двумя стеками и оценить её амортизированную стоимость?
14. Какие тесты обнаруживают ошибку перехода индекса через конец буфера?
15. Почему корректность нужно проверить до профилирования?
16. Почему вход и построение эталона выполняют вне таймера?
17. Что показывает медиана повторов и чего она не показывает?
18. Почему эксперимент не доказывает асимптотическую оценку?

10. Источники

1. Актуальная РПД дисциплины «Алгоритмы и структуры данных» для направления 03.03.01, 2026/2027 учебный год.
2. Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 4th ed. MIT Press, 2022.
3. Morin P. Open Data Structures. Open textbook. Разделы об очередях и деках.
4. Skiena S. S. The Algorithm Design Manual. 3rd ed. Springer, 2020.
5. Python Software Foundation. collections — Container datatypes; time; statistics; tracemalloc. Python 3 Documentation. <https://docs.python.org/3/> (дата обращения: 21.09.2026).

11. Лист самопроверки перед сдачей

- ☐ Контракты операций и политика ошибок указаны.
- ☐ Очередь не использует линейный сдвиг элементов.
- ☐ Инварианты `size`, `capacity` и `head` сформулированы и проверены.
- ☐ Есть тест перехода индекса через конец физического буфера.
- ☐ Худшие и амортизированные оценки обоснованы.
- ☐ Сырые времена сохранены, память измерена отдельно.
- ☐ Результаты сверены с эталоном или независимой реализацией.
- ☐ Среда, команда запуска, ограничения и внешние средства раскрыты.